Testez ce programme avec plusieurs valeurs différentes afin de voir concrètement le fonctionnement de la structure conditionnelle `if-elif-else`.

Un point très important à prendre en considération lorsque vous utilisez la structure `if-elif-else` est d'être bien conscient que les tests successifs avec la clause `if` et les clauses `elif` peuvent s'emboîter et devenir une source d'anomalies difficilement décelables ! Par exemple, si nous exécutons le code de l'exemple précédent avec la saisie du nombre 15, nous pouvons par inadvertance penser que c'est le bloc associé au test `(n%5)==0` qui sera exécuté alors que c'est bien le bloc associé au test `(n%3)==0` qui sera réellement exécuté, puisque c'est ce test qui sera rencontré en premier par l'interpréteur Python. En résumé, il faut veiller à ce que la succession des tests utilisés avec `if-elif-else` corresponde bien à la logique que vous souhaitez implémenter.

4.5 Les boucles

Les boucles permettent d'implémenter des traitements répétitifs. Un traitement est considéré comme une partie d'un programme composée d'une ou de plusieurs instructions. C'est lorsqu'un traitement doit être exécuté plusieurs fois et de manière successive que les boucles entrent en jeu.

Avec le langage Python, les boucles peuvent être implémentées en utilisant l'un des deux mots-clés `for` et `while`. Cette section montre comment construire des boucles en utilisant ces deux mots-clés et explique comment choisir entre la boucle `for` et la boucle `while`.

4.5.1 La boucle for

Nous utilisons une boucle `for` pour répéter un traitement un nombre de fois connu à l'avance, ou du moins, connu juste avant l'exécution de cette boucle.

Par exemple, analysons le programme suivant :

```
print("itération 0")
print("itération 1")
print("itération 2")
print("itération 3")
print("itération 4")
print("itération 5")
```

© Editions ENI - All rights reserved

```
print("itération 6")
print("itération 7")
print("itération 8")
print("itération 9")
print("fin du programme!")
```

En plus de la dernière instruction qui affiche un message indiquant la fin du programme, ce code est composé de dix instructions qui affichent un message différent chacun. Ces dix instructions sont formées avec la même fonction `print`. Si nous considérons que chacun des appels à cette fonction `print` est un traitement, alors les dix exécutions de ce traitement peuvent être automatisées et remplacées par une boucle. De plus, si nous observons les dix messages affichés, on voit qu'ils sont tous composés du message `itération` suivi d'un nombre allant de 0 à 9. Nous pouvons donc en conclure que nous pouvons réécrire ce programme à l'aide d'une boucle qui fera exécuter la fonction `print` un nombre de fois égal à 10 et afficher à chaque fois le message `itération` suivi d'un numéro correspondant au rang de l'exécution en cours. Connaître à l'avance le nombre de fois que le traitement de cette boucle sera exécuté justifie le choix de l'utilisation de la boucle `for` comme dans l'exemple de code suivant :

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *for-1.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
for i in range(10):
    print(f"itération {i}")
print("fin du programme!")
```

La première chose que nous pouvons observer est que les onze instructions du code précédent ont été remplacées par les trois instructions de ce dernier programme. L'exécution de ce programme donne le résultat suivant :

```
itération 0
itération 1
itération 2
itération 3
itération 4
itération 5
itération 6
itération 7
itération 8
```

```
itération 9
fin du programme!
```

La fonction `range(n)` retourne un intervalle composé des valeurs allant de `0` jusqu'à `(n-1)`. L'écriture `for i in range(10)` signifie que `i` est une variable qui va prendre tour à tour les valeurs de l'intervalle `[0,10[`, et à chaque fois que la variable `i` prend une valeur différente, le sous-bloc de code associé à la boucle `for` sera exécuté. Dans notre exemple, le sous-bloc de code associé à la boucle `for` est composé par l'unique instruction `print(f"itération {i}")`. Cette dernière instruction sera donc exécutée dix fois avec à chaque fois une valeur différente de la variable `i` comme le montre le résultat ci-dessus.

Chaque exécution d'un bloc de code associé à une boucle est appelée une itération de la boucle. Dans notre exemple, il y a donc eu dix itérations de la boucle `for` avec une valeur différente de la variable `i` à chaque itération.

À chaque fois qu'un traitement doit être exécuté de manière successive un nombre de fois connu à l'avance, la boucle `for` peut être utilisée. De plus, la variable de la boucle, comme la variable `i` dans notre exemple, permet de connaître le rang ou l'ordre de chacune des exécutions du bloc de code associé à une boucle. Cette variable associée à la boucle `for` est couramment appelée la variable d'itération, puisqu'elle permet de connaître l'ordre de l'itération en cours. La variable de la boucle est également appelée la variable compteur, puisqu'elle permet justement de compter les itérations de la boucle.

Plus haut dans ce chapitre, nous avons abordé les structures de données de type `list`. Ainsi, nous avons vu comment créer et comment modifier des listes. Le code suivant permet d'accéder aux valeurs d'une liste une par une à l'aide d'une boucle `for`.

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *for-2.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
for value in [10,11,15,18]:
    print(f"value = {value}")
print("fin du programme!")
```

© Editions ENI - All rights reserved

Dans l'exemple précédent, l'intervalle des valeurs possibles de la variable d'itération a été construit avec la fonction `range()`. Dans ce dernier code, la variable d'itération `value` prend tour à tour les valeurs de la liste `[10,11,15,18]` comme le montre le résultat suivant :

```
value = 10
value = 11
value = 15
value = 18
fin du programme!
```

Ce dernier code montre comment accéder aux valeurs d'une liste une à une à l'aide d'une boucle `for`. Cependant, souvent lorsque nous parcourons les valeurs d'une liste, nous aimerions connaître également l'ordre ou la position de chacune de ses valeurs. Pour cela, l'exemple du code ci-dessous montre comment parcourir les valeurs d'une liste tout en ayant les indices des valeurs dans cette liste :

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *for-3.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
for i, value in enumerate([10,11,15,18]) :
   print(f"à l'itération {i} value = {value}")
print("fin du programme!")
```

Ce dernier code ressemble au code précédent, sauf que cette fois nous avons utilisé la fonction `enumerate` sur la liste `[10,11,15,18]` pour former la variable d'itération de la boucle `for`. Plus précisément, ici nous avons deux variables d'itération `i` et `value` qui correspondent respectivement à l'indice d'une position de la liste `[10,11,15,18]` et à la valeur située à cette position. À la première exécution de cette boucle `for`, la variable `i` prend la valeur `0` et la variable `value` prend la valeur `10`. À la deuxième itération, la variable `i` prend la valeur `1` et la variable `value` prend la valeur `11`, et ainsi de suite comme le montre le résultat de l'exécution de ce code ci-dessous :

```
à l'itération 0 value = 10
à l'itération 1 value = 11
à l'itération 2 value = 15
à l'itération 3 value = 18
fin du programme!
```

En réalité, dans ce dernier exemple, la fonction `enumerate` retourne des tuples `(v1, v2)` dont les valeurs `v1` et `v2` sont réparties à chaque itération entre la variable `i` pour `v1` et la variable `value` pour `v2`.

Dans tous les exemples précédents, le bloc de code associé à la boucle `for` était composé d'une seule instruction. L'exemple suivant montre un cas un peu plus complexe de l'utilisation de la boucle `for` avec un sous-bloc de code composé de plusieurs instructions.

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *for-4.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
for n in [1,2,3,4,5] :
    if (n % 2)==0:
        print(f"{n} est un nombre pair")
        print(f"{n}**2 = {n**2}")
    elif (n%3)==0:
        print(f"{n} est un multiple de 3")
        print(f"{n}**3 = {n**3}")
    elif (n%5)==0:
        print(f"{n} est un multiple de 5")
        print(f"{n}**5 = {n**5}")
    else :
        print(f"{n} est un nombre impair, mais il n'est pas un
multiple de 3" \
              " ni un multiple de 5. ")
print("fin du programme!")
```

Dans ce nouvel exemple de code, nous avons repris le programme réalisé précédemment lorsque nous avons abordé la structure conditionnelle `if-elif-else`. Nous avons remplacé la partie du code où l'utilisateur était invité à saisir une valeur au clavier par une boucle `for`. Ainsi, la variable d'itération `n` prend successivement les valeurs de la liste `[1,2,3,4,5]`, puis les conditions associées aux expressions logiques `(n % 2)==0`, `(n % 3)==0` et `(n % 5)==0` sont appliquées tour à tour et un message est affiché en fonction de la condition qui est réalisée. Sinon, si aucune de ces conditions n'est réalisée, le bloc du code associé à la clause `else` est exécuté. Ce dernier code donne le résultat ci-dessous :

```
1 est un nombre impair, mais il n'est pas un multiple de 3 ni un
multiple de 5.
```

© Editions ENI - All rights reserved

```
2 est un nombre pair
2**2 = 4
3 est un multiple de 3
3**3 = 27
4 est un nombre pair
4**2 = 16
5 est un multiple de 5
5**5 = 3125
fin du programme!
```

4.5.2 La boucle for et la fonction zip

Nous avons vu que la boucle `for` est utilisée afin d'exécuter un bloc de code un nombre de fois connu à l'avance, comme par exemple parcourir l'ensemble des éléments d'une structure de données telle qu'une liste. Souvent, lorsque nous développons des programmes complexes, nous avons besoin de parcourir plusieurs listes en parallèle, c'est-à-dire exécuter un bloc de code un certain nombre d'itérations, et à chaque itération, rendre accessibles les valeurs de plusieurs listes suivant l'ordre respectif de ces valeurs dans leur liste.

Par exemple, si nous disposons des deux listes`l1=[1,2,3]` et `l2=[5,6,7]`, nous souhaitons réaliser un traitement dans une boucle en considérant à la première itération de cette boucle les deux valeurs du tuple `(1,5)`, à la deuxième itération les valeurs du tuple `(2,6)`, et enfin, les deux valeurs `(3,7)`. Pour réaliser ce type d'accès, que nous pouvons qualifier d'accès parallèle aux valeurs d'un ensemble de listes, on utilise la formidable fonction `zip`.

Avant d'utiliser la fonction `zip` conjointement avec la boucle `for`, nous allons d'abord examiner son comportement via quelques exemples simples.

Dans le premier exemple ci-dessous, nous allons appliquer la fonction `zip` sur deux listes et afficher le résultat.

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *zip-1.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
list1 = [1,2,3,4,5]
list2 = ['a','b','c','d','e']
list1_list2 = list(zip(list1,list2))
print(f"list1={list1}")
print(f"list2={list2}")
```

```
print(f"list1_list2={list1_list2}")
```

Ce code crée deux listes `list1` et `list2`, puis la fonction `zip` est appelée en lui fournissant en paramètres ces deux listes. Le résultat de l'exécution de la fonction `zip` est transformé en liste avec la fonction `list` et la liste résultante est stockée dans la variable `list1_list2`. Ce programme se termine par l'affichage des trois variables `liste1`, `liste2` et `list1_list2` comme suit :

```
list1=[1, 2, 3, 4, 5]
list2=['a', 'b', 'c', 'd', 'e']
list1_list2=[(1, 'a'), (2, 'b'), (3, 'c'), (4, 'd'), (5, 'e')]
```

On voit que le résultat de l'application de la fonction `list` sur le résultat de la fonction `zip` appliquée sur les deux listes `list1` et `list2` est une liste formée de couples dont les premiers éléments, c'est-à-dire les valeurs à gauche de ces couples, sont les valeurs de la liste `list1` et les valeurs à droite sont les valeurs de la liste `list2`.

En résumé, ces résultats montrent que la fonction `zip` permet de combiner deux listes pour former une seule structure de données composée de tuples formés à partir des valeurs des listes fournies en paramètres en respectant l'ordre des valeurs des listes d'origine.

Remarque

Dans cet exemple, nous avons appliqué la fonction `zip` sur deux listes, mais nous pouvons l'appliquer sur un nombre supérieur de listes.

Dans cet exemple, nous avons appliqué la fonction `zip` sur les deux listes `list1` et `list2` qui avaient un nombre égal d'éléments. En effet, dans chacune de ces deux listes, il y avait cinq valeurs `[1, 2, 3, 4, 5]` pour `list1` et `['a', 'b', 'c', 'd', 'e']` pour `list2`. Nous pouvons appliquer la fonction `zip` sur des listes de tailles différentes. Le résultat sera calculé en se basant uniquement sur la liste de plus petite taille, comme le montre l'exemple de code ci-dessous :

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *zip-2.py* situé dans le dossier *Code\Bases* de ce chapitre.

© Editions ENI - All rights reserved

```
list1 = [1,2]
list2 = ['a','b','c','d','e']
list1_list2 = list(zip(list1,list2))
print(f"list1={list1}")
print(f"list2={list2}")
print(f"list1_list2={list1_list2}")
```

Dans ce dernier code, `list1` a deux valeurs et `list2` a cinq valeurs et le résultat de l'application de la fonction `zip` sur ces deux listes est le suivant :

```
list1=[1, 2]
list2=['a', 'b', 'c', 'd', 'e']
list1_list2=[(1, 'a'), (2, 'b')]
```

Ces résultats montrent que les valeurs 'c', 'd' et 'e' de la liste `list2` ont été tout simplement ignorées par la fonction `zip`, puisque dans la liste `list1_list2`nous ne retrouvons que les deux couples `(1, 'a')` et `(2, 'b')` qui sont constitués des deux premières valeurs de la liste `list1` et des deux premières valeurs de la liste `list2`.

Il existe des cas où nous avons besoin d'appliquer cette fonction `zip` sur des listes de taille différente, mais où nous souhaitons également que les valeurs de la liste qui a la plus grande taille soient incluses dans les résultats. Évidemment, dans ces cas-là, les dernières valeurs de la liste de plus grande taille n'auront pas de valeurs correspondantes dans les listes de plus petite taille. La solution est d'affecter des valeurs par défaut à la place des valeurs manquantes dans les listes de plus petite taille, comme le montre l'exemple de code suivant :

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *zip-3.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
from itertools import zip_longest

list1 = [1,2]
list2 = ['a','b','c','d','e']
list1_list2 = list(zip_longest(list1,list2, fillvalue='X'))

print(f"list1={list1}")
print(f"list2={list2}")
print(f"list1_list2={list1_list2}")
```

Ce code commence par l'instruction `from itertools import zip_longest` qui permet d'importer la fonction `zip_longest` à partir du module `itertools`.

Comme nous l'avons indiqué précédemment, un module peut être vu comme un regroupement de fonctionnalités, développées afin de répondre à des besoins spécifiques. Nous allons découvrir bien d'autres modules au fur et à mesure de l'avancement de ce livre.

Le module `itertools` n'est pas chargé par défaut par l'interpréteur Python, et c'est bien pour cette raison que nous avons utilisé cette première instruction qui permet de charger la fonction `zip_longest` pour ainsi la rendre disponible à notre programme.

Le reste de ce programme ressemble au programme précédent, mais cette fois, au lieu d'utiliser la fonction `zip`, nous avons utilisé la fonction `zip_longest()` en lui fournissant les deux arguments `list1` et `list2` qui correspondent aux deux listes à partir desquelles les tuples vont être construits et un troisième argument `fillvalue='X'` qui indique que la valeur `X` sera utilisée pour compléter les tuples si des valeurs sont manquantes. Ce programme affiche les résultats suivants :

```
list1=[1, 2]
list2=['a', 'b', 'c', 'd', 'e']
list1_list2=[(1, 'a'), (2, 'b'), ('X', 'c'), ('X', 'd'),
('X', 'e')]
```

Vous remarquez que cette fois la liste `list1_list2` contient cinq valeurs et non pas que deux valeurs comme dans l'exemple précédent et qu'à partir du troisième tuple de cette liste, l'élément situé à gauche correspond à la valeur `X` qui a été indiquée avec l'argument `fillvalue`.

Avant de montrer comment combiner l'utilisation de la fonction `zip` avec la boucle `for`, il est intéressant de savoir que cette fonction `zip` peut être utilisée pour réaliser la fonction inverse de ce que nous avons vu lors des exemples précédents. C'est-à-dire que nous pouvons par exemple fournir à la fonction `zip` une liste de couples et obtenir en retour deux tuples séparés dont le premier tuple contiendrait toutes les valeurs situées à gauche des couples d'origine et le deuxième tuple contiendrait les valeurs situées à droite des couples d'origine comme dans l'exemple de code suivant.

© Editions ENI - All rights reserved

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *zip-4.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
list1 = [1,2,3,4,5]
list2 = ['a','b','c','d','e']
list1_list2 = list(zip(list1,list2))
print(f"list1={list1}")
print(f"list2={list2}")
print(f"list1_list2={list1_list2}")

tuple1,tuple2 = zip(*list1_list2)
print(f"tuple1={tuple1}")
print(f"tuple2={tuple2}")
```

Ce code crée la liste `list1_list2` en appliquant la fonction `zip` sur les deux listes `list1` et `list2`. Puis, avec l'instruction `tuple1,tuple2 = zip(*list1_list2)`, les deux tuples `tuple1` et `tuple2` sont créés à partir des couples qui seraient stockés dans la liste `list1_list2`. Le résultat de l'exécution de ce code est le suivant :

```
list1=[1, 2, 3, 4, 5]
list2=['a', 'b', 'c', 'd', 'e']
list1_list2=[(1, 'a'), (2, 'b'), (3, 'c'), (4, 'd'), (5, 'e')]
tuple1=(1, 2, 3, 4, 5)
tuple2=('a', 'b', 'c', 'd', 'e')
```

On voit que `tuple1` contient toutes les valeurs situées à gauche des couples de la liste `list1_list2` alors que `tuple2` contient toutes les valeurs situées à droite de cette même liste.

La fonction `zip` est souvent utilisée avec la boucle `for` dans le but de parcourir plusieurs listes en parallèle. Dans l'exemple de code suivant, la fonction `zip` est utilisée pour parcourir les deux listes `list1` et `list2` en parallèle.

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *for-zip-1.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
list1 = [1,2,3,4,5]
list2 = [6,7,8,9,10]
for v1,v2 in zip(list1, list2):
    print(f"{v1} + {v2} = {v1+v2}")
```

Ce programme combine la puissance de la fonction `zip` avec la boucle `for` afin de parcourir deux à deux les valeurs des deux listes `list1` et `list2`. Dans cet exemple, le bloc de code associé à la boucle `for` se résume à l'unique appel de la fonction `print` qui permet d'afficher le résultat de l'expression `v1+v2`, avec `v1` une des valeurs de la liste `list1` et `v2` la valeur qui a la même position que `v1`, mais dans la liste `list2`. Le résultat de ce programme est le suivant :

```
1 + 6 = 7
2 + 7 = 9
3 + 8 = 11
4 + 9 = 13
5 + 10 = 15
```

On voit que les éléments des deux listes `list1` et `list2` ont été additionnés deux à deux suivant leur position dans leurs listes respectives.

Il est encore plus intéressant de parcourir les valeurs de plusieurs listes en parallèle et d'avoir le rang de chaque itération comme dans l'exemple du code suivant :

```
list1 = [1,2,3,4,5]
list2 = [6,7,8,9,10]
for i, (v1,v2) in enumerate(zip(list1, list2)):
   print(f"{i}: {v1} + {v2} = {v1+v2}")
```

Ce code ressemble beaucoup au code précédent, sauf qu'ici nous avons ajouté la variable `i` qui permet de connaître le rang de l'itération en cours. Cette variable `i` est renseignée par la fonction `enumerate` qui est elle-même appliquée sur le résultat de la fonction `zip`. Le résultat de ce programme est le suivant :

```
0: 1 + 6 = 7
1: 2 + 7 = 9
2: 3 + 8 = 11
3: 4 + 9 = 13
4: 5 + 10 = 15
```

On voit que la variable `i` indique bien l'ordre de l'itération en cours !

© Editions ENI - All rights reserved

4.5.3 La boucle while

La boucle `for` est souvent utilisée lorsque le nombre d'itérations à effectuer est connu à l'avance. La boucle `while` sert à réaliser des itérations dont le nombre n'est pas connu à l'avance. La boucle `while` est utilisée avec une ou plusieurs conditions. Ces conditions servent justement pour décider de l'arrêt ou de la continuité de l'exécution des itérations de la boucle `while`. L'exemple suivant illustre l'utilisation de la boucle `while`.

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *while.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
n = int(input("Veuillez saisir un chiffre entre 5 et 10 ? "))
while n in range(5,11):
    print(f"{n}**2 = {n**2}")
    n = int(input("Veuillez saisir un chiffre entre 5 et 10 ? "))
print("fin du programme!")
```

Ce programme commence par inviter l'utilisateur à saisir un chiffre entre 5 et 10 et stocke dans la variable `n` le résultat de la transformation de la chaîne de caractères saisie par l'utilisateur en un nombre entier en utilisant la fonction `int()`. La condition de la boucle `while` contrôle si la valeur de cette variable `n` est bien dans l'intervalle `[5,10]`. Si la valeur de la variable `n` est dans cet intervalle, alors le corps de la boucle `while` est exécuté, sinon le programme passe à l'instruction située après le bloc du code de cette boucle `while` et qui, dans notre exemple, correspond à la dernière instruction qui affiche le message `fin du programme !`. Dans la mesure où le chiffre saisi est dans l'intervalle `[5,10]`, le carré de ce chiffre sera calculé et affiché, puis de nouveau l'utilisateur sera invité à saisir un nouveau chiffre.

Le résultat suivant montre un exemple d'exécution de ce programme :

```
Veuillez saisir un chiffre entre 5 et 10 ? 5
5**2 = 25
Veuillez saisir un chiffre entre 5 et 10 ? 6
6**2 = 36
Veuillez saisir un chiffre entre 5 et 10 ? 5
5**2 = 25
Veuillez saisir un chiffre entre 5 et 10 ? 6
6**2 = 36
Veuillez saisir un chiffre entre 5 et 10 ? 7
```

```
7**2 = 49
Veuillez saisir un chiffre entre 5 et 10 ? 7
7**2 = 49
Veuillez saisir un chiffre entre 5 et 10 ? 8
8**2 = 64
Veuillez saisir un chiffre entre 5 et 10 ? 9
9**2 = 81
Veuillez saisir un chiffre entre 5 et 10 ? 10
10**2 = 100
Veuillez saisir un chiffre entre 5 et 10 ? 11
fin du programme!
```

On voit que tant que la valeur saisie par l'utilisateur est dans l'intervalle `[5,10]`, le bloc de la boucle `while` sera exécuté. De ce fait, le nombre d'exécutions de cette boucle ne peut pas être connu à l'avance et c'est dans cette dernière constatation que l'utilisation de la boucle `while` est justifiée par rapport à l'utilisation de la boucle `for`.

4.5.4 Contrôler les boucles avec break

Souvent, lorsque nous développons une boucle, avec `for` ou `while`, nous avons besoin d'arrêter le déroulement de cette boucle de façon prématurée, car un événement particulier s'est produit.

Dans le code suivant, nous avons utilisé la boucle `for` afin de contrôler si le nombre saisi par l'utilisateur est un nombre premier ou pas.

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *for-break.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
n = int(input("Veuillez saisir un nombre > 1 ? "))
prime=True
if n < 2:
   prime = False
else:
   for i in range(2,n):
       if (n%i)==0:
           prime=False
           break
if prime :
   print(f"Le nombre {n} est un nombre premier")
else :
```

© Editions ENI - All rights reserved

```
    print(f"Le nombre {n} n'est pas un nombre composé")
print("fin du programme!")
```

Pour contrôler si le nombre saisi par l'utilisateur est premier ou pas, nous avons divisé ce nombre dans une boucle `for` sur les nombres inférieurs à lui et supérieurs à 1, mais dès qu'un diviseur est trouvé, la variable `prime` sera affectée à `False` pour mémoriser le fait qu'un diviseur a été trouvé et cette boucle `for` s'arrête en utilisant la clause `break`. Dans ce programme, si le nombre `n` est strictement inférieur à 2, il sera considéré comme étant un nombre composé, donc pas un nombre premier.

Dans cet exemple, nous avons utilisé la boucle `for`, car nous savons que le nombre maximal d'itérations est fixé par le nombre `(n-1)`. En revanche, nous savons que si un diviseur du nombre `n` existe, alors il ne sera pas nécessaire d'exécuter les itérations restantes de cette boucle, puisqu'un seul diviseur supérieur à 1 et inférieur à `n` suffit pour déclarer qu'un nombre n'est pas premier, d'où la justification d'arrêter le déroulement des itérations avec `break`.

4.5.5 Contrôler les boucles avec continue

Avec la clause `continue`, on peut décider d'arrêter l'exécution d'une itération d'une boucle, mais pas les prochaines itérations ! En effet, dans certaines conditions, on aimerait interrompre l'exécution de l'itération en cours pour continuer à partir de la prochaine itération.

Le code suivant permet d'afficher le carré des chiffres pairs entre 1 et une valeur saisie au clavier.

▶ Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *for-continue.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
n = int(input("Veuillez saisir un nombre > 1 ? "))

for i in range(1, n+1):
    if (i%2)!=0 :
        continue
    print(f"{i}**2 = {i**2}")

print("fin du programme!")
```

La boucle `for` de ce code parcourt les valeurs de 1 jusqu'à `n` et à chaque fois qu'une de ces valeurs est impaire, l'instruction `print` située au sein du bloc de code de cette boucle `for` sera ignorée et une nouvelle itération sera considérée.

L'exécution de ce code avec la valeur 10 saisie au clavier donne le résultat ci-dessous :

```
Veuillez saisir un nombre > 1 ? 10
2**2 = 4
4**2 = 16
6**2 = 36
8**2 = 64
10**2 = 100
fin du programme!
```

On voit que les valeurs 3, 5, 7 et 9 ont été ignorées par le programme ! En effet, pour ces valeurs, l'instruction `continue` permet de repartir du début de la boucle `for` en ignorant tout ce qui se trouve après cette clause `continue`.

Remarque

Bien que les clauses `break` et `continue` peuvent s'avérer intéressantes et justifiées, il faut les éviter autant que possible, car elles peuvent introduire de la complexité qui nuit à la bonne lisibilité des programmes. En résumé, `break` et `continue` sont utiles, mais à utiliser avec modération.

4.6 Les fonctions

Nous avons vu que les boucles permettent d'automatiser un traitement dont lequel un bloc de code doit être exécuté plusieurs fois de façon successive. En revanche, comment faire lorsqu'un bloc de code doit être exécuté plusieurs fois, mais pas forcément de manière successive ?

Lorsque les instructions d'un bloc de code apparaissent plusieurs fois dans des endroits différents de notre programme, nous pouvons tout simplement factoriser ces instructions dans une seule et même partie de ce programme. Cette partie du programme où les instructions de plusieurs blocs de code ont été factorisées est appelée une procédure ou un sous-programme.

© Editions ENI - All rights reserved